МИНОБРНАУКИ РОССИИ
РГУ НЕФТИ И ГАЗА (НИУ) ИМЕНИ И.М. ГУБКИНА

Факультет **Комплексной безопасности ТЭК**
Кафедра **Безопасности информационных технологий**

ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ

по дисциплине Языки, методы и технологии программирования

на тему Разработка паттерна для обеспечения безопасности приложения при угрозе использования уязвимых версий программного обеспечения

ДАНО студенту **Исаева Яна Константиновна** (фамилия, имя, отчество в дательном падеже) группы **КС-23-18** (номер группы)

Содержание работы:

1. Введение
2. Анализ предметной области
3. Разработка паттерна
4. Разработка микросервисов
5. Тестирование и мониторинг микросервисов на кластере
6. Заключение
7. Приложение

Исходные данные для выполнения работы:

1. Информация о предметной области из открытых источников

Рекомендуемая литература:

1. Положение о курсовом проектировании в РГУ нефти и газа (НИУ) имени И.М. Губкина (приказ от 11.11.2021 № 398)
2. Docker Compose Documentation [Электронный ресурс]: сайт. – Режим доступа: https://docs.docker.com/compose/
3. Kubernetes Documentation [Электронный ресурс]: сайт. – Режим доступа: https://kubernetes.io/docs/home/

Графическая часть:

1. Модель угрозы; 2. Механизм защиты; 3. Архитектура паттерна; 4. UML диаграммы; 5. Коды микросервисов; 6. Витрины с результатами мониторинга и тестирования микросервисов на кластере

Требования к представлению результатов:

Электронная версия

Руководитель: д.т.н. (уч.степень) профессор (должность) (подпись) Корнеев Николай Владимирович (фамилия, имя, отчество)

Задание принял к исполнению: студент (подпись) Исаева Яна Константиновна (фамилия, имя, отчество)

МИНОБРНАУКИ РОССИИ

РГУ НЕФТИ И ГАЗА (НИУ) ИМЕНИ И.М. ГУБКИНА

Факультет **Комплексной безопасности ТЭК**

Кафедра **Безопасности информационных технологий**

Оценка комиссии: Рейтинг:

Подписи членов комиссии:

(подпись) (фамилия, имя, отчество)

(подпись) (фамилия, имя, отчество)

(дата)

КУРСОВАЯ РАБОТА

по дисциплине Языки, методы и технологии программирования

на тему Разработка паттерна для обеспечения безопасности приложения при угрозе использования уязвимых версий программного обеспечения

«К ЗАЩИТЕ»

Профессор, д.т.н. Корнеев Н.В.

(должность, ученая степень; фамилия, и.о.)

(подпись)

(дата)

ВЫПОЛНИЛ:

Студент группы **КС-23-18**

(номер группы)

Исаева Яна Константиновна

(фамилия, имя, отчество)

(подпись)

(дата)

Москва, 20 25

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3

1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ 5

1.1. Модель угрозы 5

1.2. Механизм защиты 8

2. РАЗРАБОТКА ПАТТЕРНА 11

2.1. Архитектура паттерна 11

2.2. UML-диаграммы 12

3. РАЗРАБОТКА МИКРОСЕРВИСОВ 17

3.1. Функции микросервисов 17

3.2. Развертывание микросервиса 28

4. МОНИТОРИНГ МИКРОСЕРВИСОВ 32

4.1. Тестирование микросервисов 32

4.2. Мониторинг микросервисов 34

ЗАКЛЮЧЕНИЕ 36

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ 38

ВВЕДЕНИЕ

В настоящее время программное обеспечение максимально внедрилось в жизнь человека, если можно так сказать. Оно играет ключевую роль в работе компьютеров, специализированных систем, а также мобильных устройств. Фактически, программное обеспечение стало невидимым, но жизненно необходимым инфраструктурным слоем современной цивилизации, отвечающим за эффективность, безопасность и комфорт повседневно жизни человека. Однако из-за стремительного роста технологий и разработки, сложности программных комплексов и широкого использования сторонних компонентов, различных библиотек или фреймворков, произошел резкий скачок уязвимостей. Согласно данным из репозиториев, таких как National Vulnerability Database (NVD), ежегодно фиксируются десятки тысяч новых уязвимостей.

Серьезной угрозой выступают уязвимости в программном обеспечении или его компонентах, поскольку эти недостатки могут стать удобной почвой для различных кибератак. В следствие чего возникает острая необходимость не только вовремя реагировать на возникающие инциденты, но и внедрять специализированные механизмы, интегрируемые в процессы разработки и эксплуатации. Наилучшем и наиболее эффективным методом решения этой проблемы является разработка и создание паттерна, учитывающего принципы безопасности в структуре программного обеспечения.

Данная тема актуальна, поскольку современная цифровая инфраструктура тесно связана с программным обеспечением. Ведь количество уязвимостей в используемых программных компонентах активно увеличивается, тем самым требуя активного внедрения принципов защиты на уровне архитектуры приложений.

Объектом исследования является процесс обеспечения безопасности программного обеспечения, направленный на минимизацию рисков, которые связаны с уязвимостью программного обеспечения.

В качестве предмета исследования выступает архитектурный паттерн, как способ минимизации последствий использования программного обеспечения с

уязвимостями в рамках жизненного цикла приложения.

Целью работы является разработка архитектурного паттерна, обеспечивающего безопасность приложения при использовании уязвимых версий программного обеспечения с помощью внедрения механизмов изоляции и ограничения прав доступа для потенциально опасных компонентов.

Для достижения поставленной цели необходимо выполнить следующие задачи:

1. Провести анализ существующей ситуации в области безопасности мобильных устройств.
2. Разработать архитектуру предлагаемого паттерна.
3. Реализовать разработанный паттерн.
4. Провести тщательное тестирование паттерна.
5. Определить пути его внедрения.

Работа имеет практическую ценность, поскольку предлагает готовое к использованию архитектурное решение – паттерн безопасности. Внедрение данного паттерна повышает отказоустойчивость приложений и минимизирует ущерб от атак до обновления опасных зависимостей, напрямую уменьшая риски и репутационные потери.

Научная и практическая новизна исследования заключается в разработке специализированного архитектурного паттерна безопасности, ориентированного на комплексное решение проблемы использования уязвимых версий программного обеспечения в современных приложениях. Инновационность подхода заключается в интеграции принципов изоляции уязвимых элементов, тщательной верификации данных и управления доступом в единую метрику, разработанную для работы с потенциально надежными зависимостями. Разработанный паттерн будет, так скажем, выполнять концепцию «безопасного обертывания» для потенциально ненадежных компонентов. Что позволит минимизировать риски немедленного обновления уязвимой версии программного обеспечения после проведения атаки.

1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1. Модель угрозы

Сценарий 1: Нарушение доступности через эксплуатацию уязвимостей ПО

Цель злоумышленника - вызвать полный или частичный отказ в обслуживании веб- приложения путем исчерпания вычислительных ресурсов через эксплуатацию известных уязвимостей в устаревших версиях программных компонентов.

Источник атаки – специально сформированный вредоносный HTTP-запрос, эксплуатирующий конкретную уязвимость в программном компоненте. Запрос является корректным с точки зрения сетевого протокола, но содержит в своем теле, заголовках или параметрах специфическую последовательность данных, активирующую дефектный код в целевой системе.

Объекты, которые могут стать жертвами атаки:

1. Веб-серверы с уязвимыми версиями;
2. Системы управления базами данных (MySQL, PostgreSQL);
3. Интерпретаторы серверных языков (PHP, Python);

Принцип атаки заключается в использовании уязвимостей в реализации программного обеспечения. Эти уязвимости приводят к избыточному потреблению вычислительных ресурсов при обработке специально сформированных, но технически корректных запросов.

Сценарий атаки:

1. Злоумышленник проводит сканирование целевого веб-приложения для выявления версий используемого ПО (веб-сервера nginx/Apache, интерпретатора PHP, библиотек).
2. На основании обнаруженных уязвимых версий разрабатывается или адаптируется эксплойт, направленный на создание условий отказа в обслуживании.
3. Запускается поток специально сформированных HTTP/HTTPS запросов, которые сначала эксплуатируют уязвимости управления памятью для вызывания ее утечки, затем используют ошибки обработки протоколов для создания бесконечных циклов. И в итоге задействуют дефекты

многопоточности для блокировки системных ресурсов.

4. Каждый успешный запрос вызывает непропорционально высокую нагрузку на сервер, что приводит к каскадному отказу.

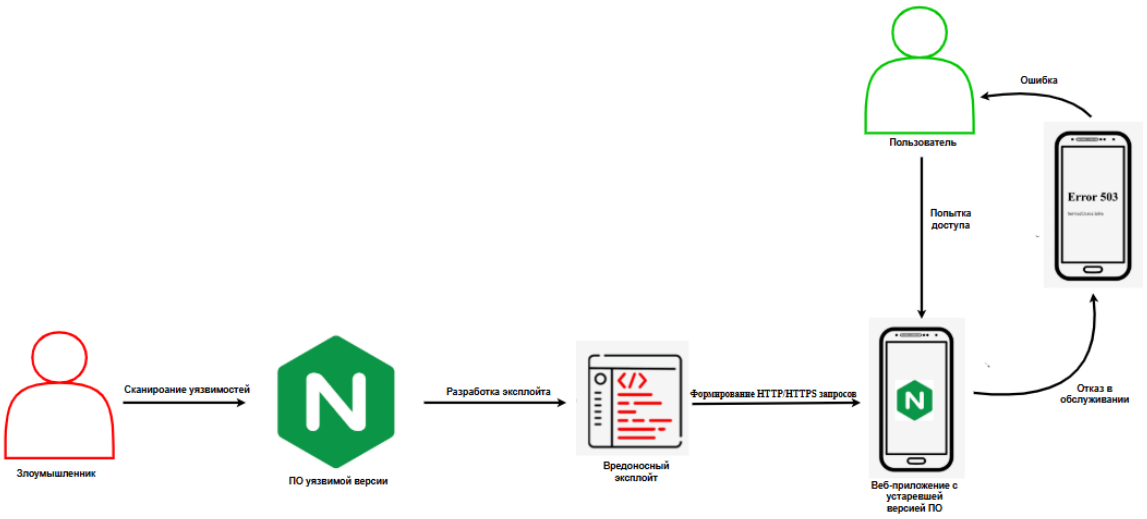


Рисунок 1 – Сценарий нарушения доступности через эксплуатацию уязвимостей ПО

Сценарий 2. Нарушение целостности с использованием SQL-запроса.

Цель злоумышленника получить незаконный доступ к данным, чтобы изменить, подделать, украсть или же уничтожить их. Для этого он использует слабые места (уязвимости) в устаревших компонентах программного обеспечения.

Источник атаки – вредоносный код (HTTP-запрос с инъекцией, модифицированный файл, специально сформированный сетевой пакет), разработанный для эксплуатации конкретной уязвимости в программном обеспечении.

Возможные объекты атаки:

1. Системы управления базами данных с известными уязвимостями.
2. Серверы приложений (Apache Tomcat, JBoss).
3. Библиотеки

Принцип атаки строится на использовании уязвимостей в программном обеспечении, которые нарушают основные принципы безопасности, а именно

разделение данных и инструкций, аутентификацию и проверку корректности ввода. Данная атака возможна, если система некорректно разделяет пользовательские данные и команды. Уязвимая версия программного обеспечения не проводит достаточной проверки пользовательского ввода, на что и делает упор атака. Все приводит к передаче данных в качестве исполняемых инструкций.

Сценарий атаки:

1. Злоумышленник анализирует веб-приложение, отправляя пробные запросы с метасимволами в поля форм, чтобы выявить страницы, возвращающие ошибки базы данных, что подтверждает возможность SQL-инъекции и помогает определить тип СУБД.
2. Используя различные техники, злоумышленник определяет структуру базы данных, например, имена таблиц, столбцов.
3. Далее эксплуатируя уязвимость, атакующий обходит аутентификацию, подделав учетные данные, извлекает конфиденциальные данные (хеши паролей), тем самым получая доступ к данным.
4. После получения доступа атакующий изменяет существующие записи, удаляет или крадет критически важные данные и добавляет новые записи, поддельные.

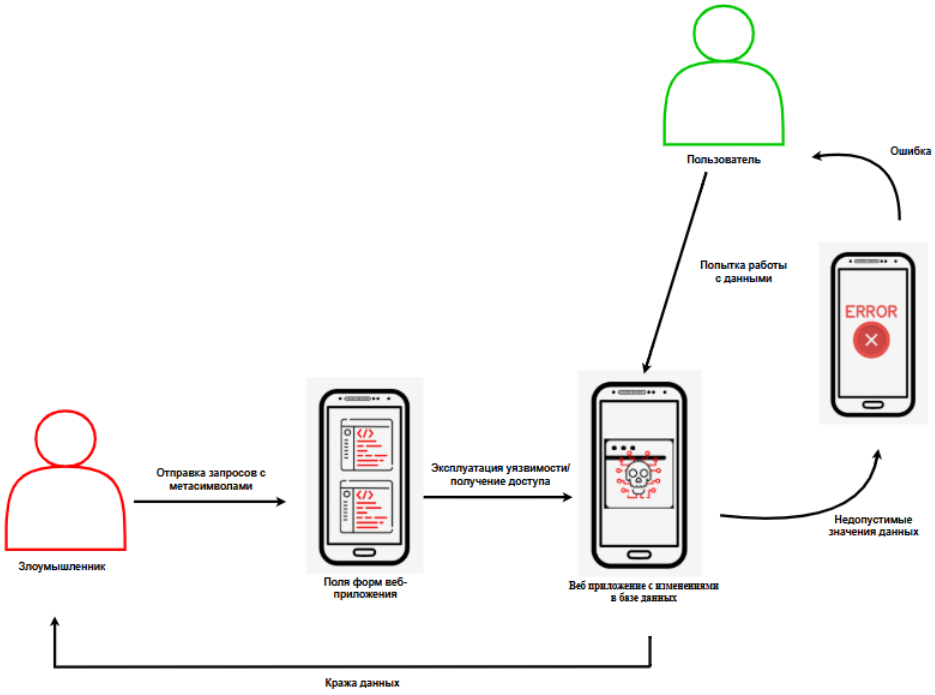


Рисунок 2 – Сценарий нарушение целостности с использованием SQL-запроса

1.2. Механизм защиты

После разработки моделей угроз необходимо перейти к формированию системы мер, направленных на предотвращение реализации актуальных угроз и снижение рисков для доступности и целостности информации.

Защитный механизм формируется через комплекс согласованных организационных и технических мер, направленных на нейтрализацию всех компонентов модели угроз, включая уязвимые объекты, источники опасности, а также методы реализации атак и их последствия.

Сценарий 1: Нарушение доступности через эксплуатацию уязвимостей ПО

Разработанный механизм защиты распознает и останавливает атаку до того, как вредоносный запрос достигнет и нагрузит уязвимый компонент приложения. Паттерн безопасности выступает в качестве фильтра, который учится на инцидентах, обновляет свои правила и адаптирует политики зашиты в реальном времени, предотвращая каскадный отказ системы.

1. Злоумышленник через специально сформированный HTTP-запрос пытается вызвать отказ в обслуживании веб-приложения, эксплуатируя уязвимость в устаревшем веб-сервере;
2. Запрос от клиента перенаправляется через API Gateway к сервису сканирования уязвимостей, который перехватывает обращение и проверяет его на соответствие сигнатурам известных эксплойтов.
3. Проверяет содержимое запроса и его потенциальная опасность для версий ПО, используемых в системе, в случае обнаружения совпадения с сигнатурой атаки выполняется немедленная блокировка запроса.
4. Сервис регистрирования инцидентов регистрирует данный инцидент, обновляя правила фильтрации и инициирует автоматическое применение обновления для уязвимого компонента, предотвращая дальнейшие попытки атаки.

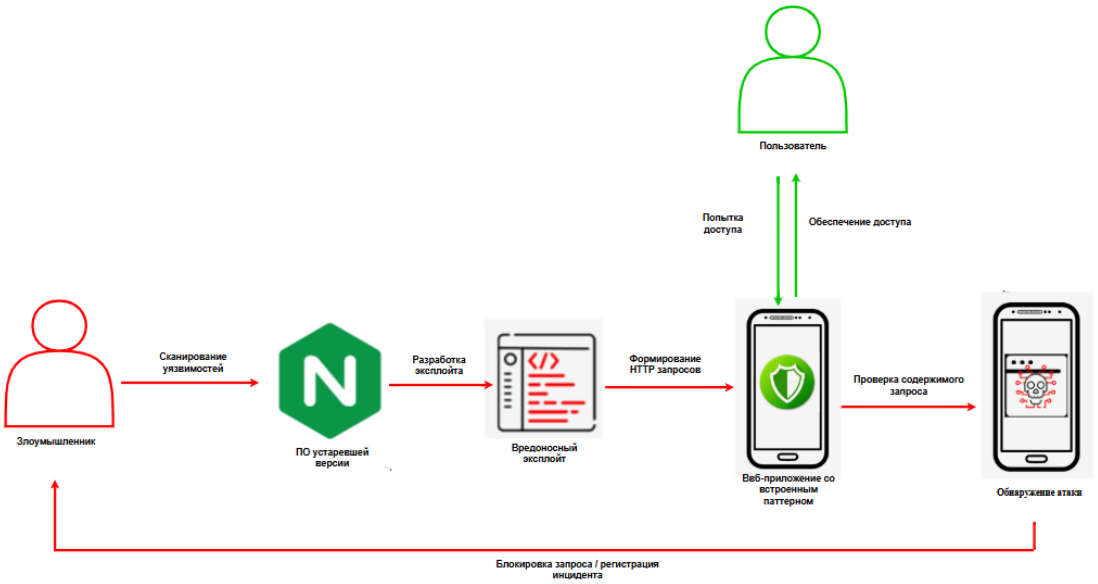


Рисунок 3 – Сценарий нарушения доступности через эксплуатацию уязвимостей ПО

Сценарий 2. Нарушение целостности с использованием SQL-запроса.

Механизм защиты по своей структуре использует специальные меры (валидацию, минимальные привилегии), устраняя саму возможность инъекции и включает в себя активный мониторинг и реагирование, нейтрализующие угрозу, если специальные меры по какой-то причине оказались недостаточными. В итоге, паттерн не просто блокирует вредоносный запрос, но и создает такую среду, где для SQL-инъекции не остается уязвимых точек входа.

1. Злоумышленник через поле ввода веб-формы пытается внедрить SQL-код в запрос к базе данных, используя уязвимость в не обновленной версии ПО или ошибку валидации;
2. Пользовательские данные из запроса перенаправляются через API Gateway к сервису валидации безопасности, который перехватывает ввод и проверяет его на наличие опасных символов и шаблонов инъекций;
3. Проверяются входные данные и их допустимость в контексте ожидаемого формата, при условии обнаружения попытки SQL-инъекции выполняется блокировка запроса.

4. Сервис контроля доступа фиксирует попытку несанкционированного доступа, а сервис мониторинга инцидентов логирует инцидент для последующего аудита.

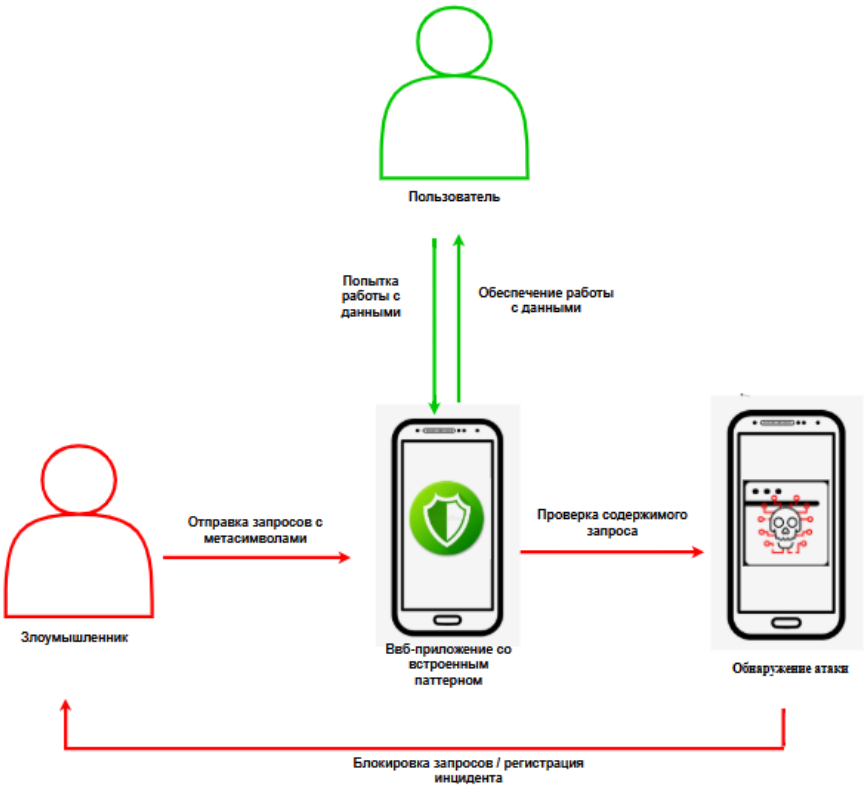


Рисунок 4 – Сценарий нарушение целостности с использованием SQL-запроса

2. РАЗРАБОТКА ПАТТЕРНА

2.1. Архитектура паттерна

В разрабатываем паттерне безопасности реализована многоуровневая система защиты, основанная на пяти специализированных микросервисах, каждый из которых выполняет строго определенные функции обеспечения безопасности веб- приложения. Для реализации паттерна защиты веб-приложений от угроз, связанных с уязвимыми версиями программного обеспечения, использован следующий стек технологий: Python, Go, Docker, Kubernetes, PostgreSQL, Prometheus, Grafana. Паттерн безопасности построен на основе микросервисной архитектуры (Microservice Architecture, MA), состоящей из пяти специализированных сервисов: Vulnerability Scanner для сканирования уязвимостей, Security Validator для валидации данных, Access Controller для управления доступом Incident Monitor для мониторинга инцидентов и сервис логироания.

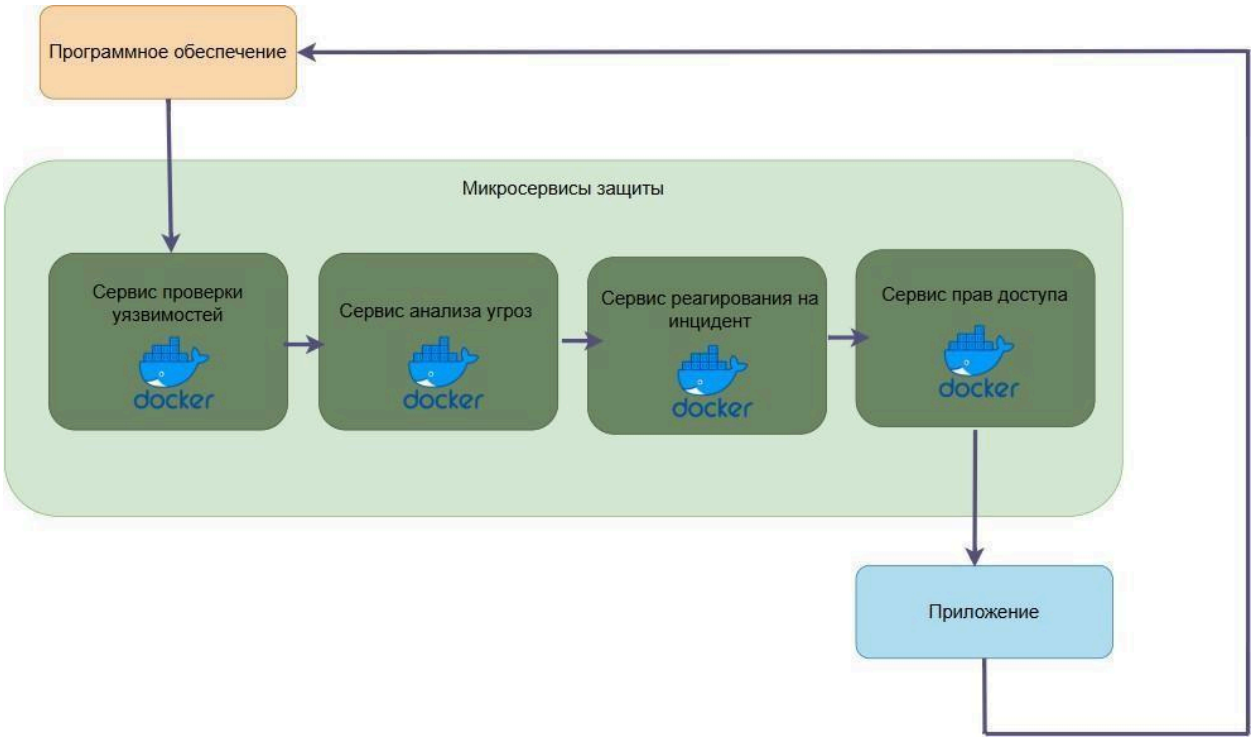


Рисунок 5 – Микросервисная архитектура паттерна обеспечения доступности и целостности ПО с учетом уязвимостей в нем

API Gateway выступает в качестве единой точки входа и осуществляет первоначальную маршрутизацию всех входящих запросов, обеспечивая также базовую проверку корректности запросов и балансировку нагрузки.

Vulnerability Scanner осуществляет проактивное сканирование всех компонентов веб-приложения. Он анализирует версии используемого ПО, сверяет их с актуальными базами уязвимостей CVE и BDU, выявляет потенциально опасные конфигурации и оценивает уровень риска, обеспечивая своевременное обнаружение угроз до их эксплуатации.

Security Validator отвечает за проверку и очистку всех входных данных. Этот сервис выполняет многоуровневую валидацию, включая проверку типов данных, экранирование специальных символов и обнаружение попыток SQL-инъекций, XSS-атак и других методов нарушения целостности данных.

Access Controller управляет системой разграничения доступа. Он реализует строгую аутентификацию пользователей, проверяет авторизацию на основе политик RBAC, контролирует сессии и обеспечивает соблюдение принципа минимальных привилегий для всех операций с данными.

Incident Monitor является центральным узлом системы реагирования. Он собирает события от всех компонентов, коррелирует их для выявления сложных атак, автоматически классифицирует инциденты по степени критичности и инициирует заранее определенные сценарии автоматического реагирования.

Logging Service обеспечивает централизованное логирование всей активности системы. Он собирает, структурирует, индексирует и сохраняет журналы событий, аудиторские записи и метрики производительности со всех сервисов, создавая единую основу для расследования инцидентов, аудита безопасности и анализа эффективности защитных мер.

Данная архитектура обеспечивает независимое масштабирование каждого сервиса, высокую отказоустойчивость и глубокую интеграцию с современными системами мониторинга и аналитики угроз.

2.2. UML-диаграммы

Перейдем к диаграммам классов и взаимодействия. На рисунке 6 изображена UML-диаграмма последовательности для разработанного паттерна безопасности. На ней можно наглядно увидеть динамическое взаимодействие между компонентами паттерна безопасности в хронологическом порядке при

обработке входящего запроса.

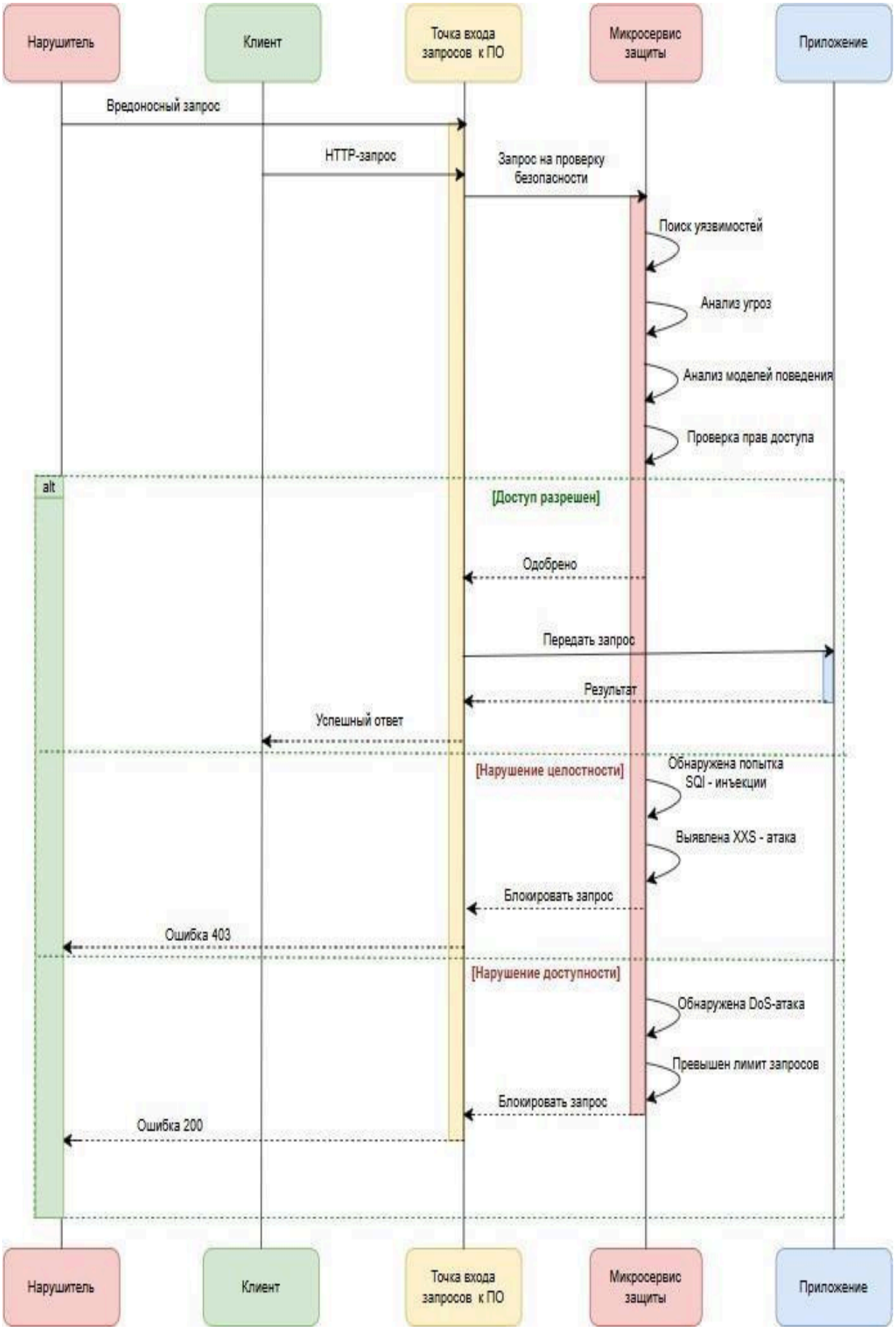


Рисунок 6 – UML-диаграмма последовательности паттерна безопасности.

На UML-диаграмме состояний микросервисов (рисунок 7) показывается жизненный цикл каждого микросервиса паттерна безопасности и переходы между специальными классами. Каждый класс выполняет свою персональную задачу, тем самым обеспечивая защиту от возможных атак.

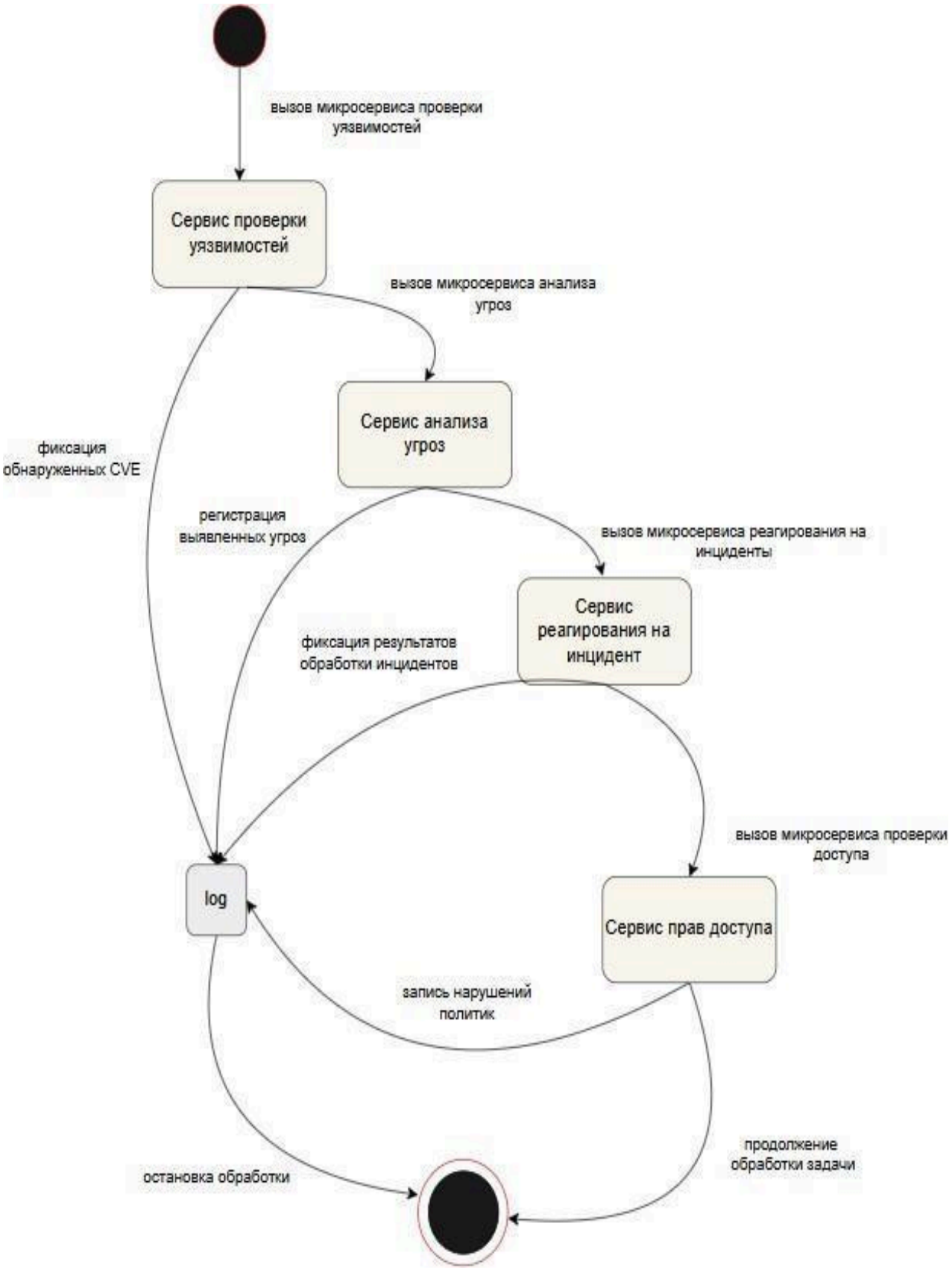


Рисунок 7 – UML-диаграмма состояния паттерна безопасности.

Рассмотрев диаграмму деятельности (рисунок 8), отображающую весь процесс функционала паттерна, можно увидеть подробный алгоритм обработки запроса с акцентом на процессы защиты от угроз, связанных с уязвимыми версиями ПО. Основное внимание уделяется последовательности проверок безопасности. Диаграмма показывает, как запрос постепенно проходит через каждый микросервис, в результате либо обрабатываясь, либо подбрасываясь в качестве вредоносного.

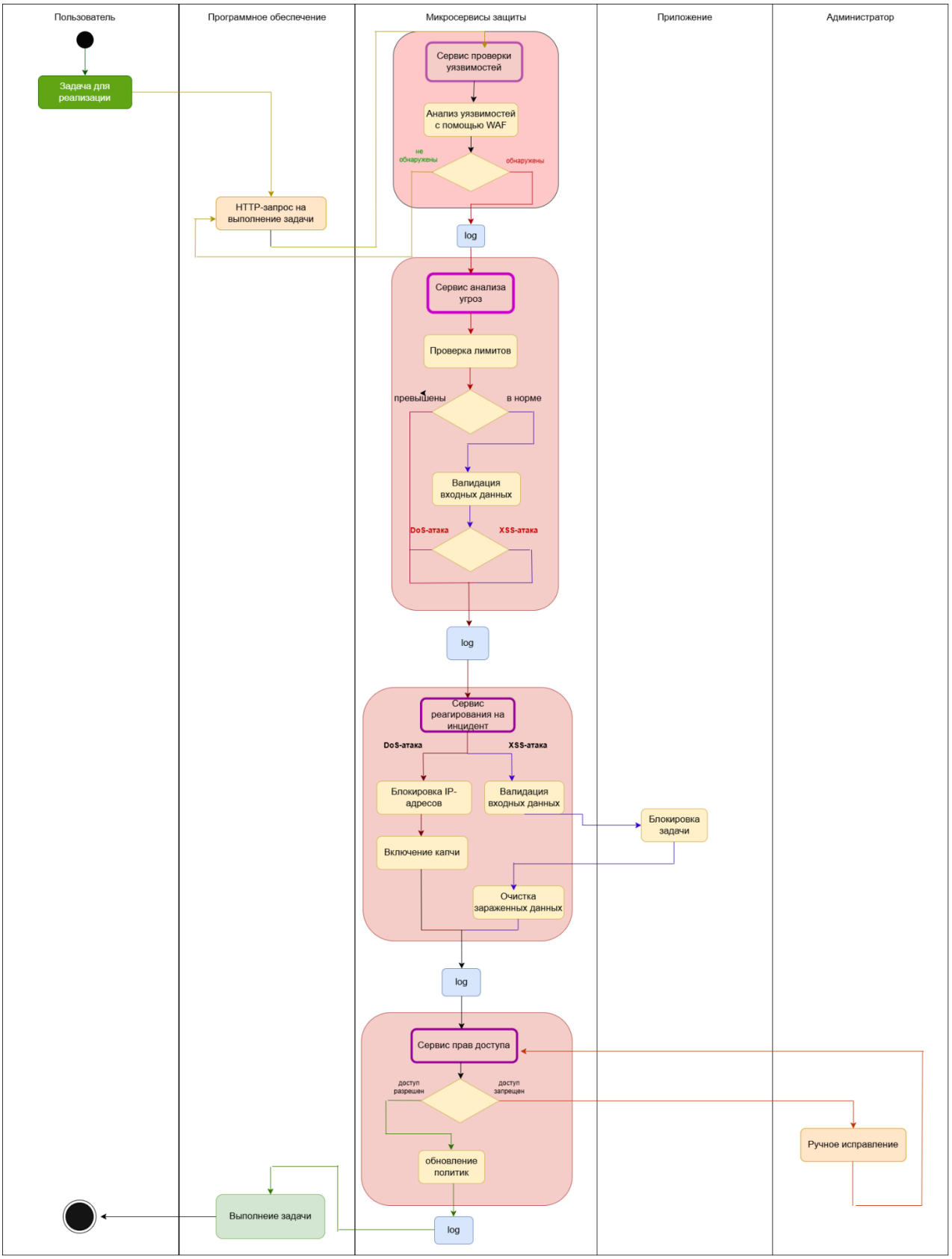


Рисунок 8 – UML-диаграмма деятельности паттерна безопасности

3. РАЗРАБОТКА МИКРОСЕРВИСОВ

3.1. Функции микросервисов

1. Микросервис Vulnerability Scanner (сканирование уязвимостей).

Данная функция выполняет критически важную операцию проверки, попадает ли конкретная версия программного компонента в диапазон уязвимых версий, указанный в базе данных уязвимостей CVE (Common Vulnerabilities and Exposures). Функция принимает два строковых параметра: current_version — текущая проверяемая версия ПО, и affected_range — строковое описание диапазона уязвимых версий в формате, принятом в базах CVE.

```
def is_version_vulnerable(current_version: str, affected_range: str) -> bool:
    from packaging import version
    try:
        current = version.parse(current_version)
        if affected_range.startswith("<="):
            max_ver = version.parse(affected_range[2:])
            return current <= max_ver
        elif affected_range.startswith("<"):
            max_ver = version.parse(affected_range[1:])
            return current < max_ver
        elif affected_range.startswith(">="):
            min_ver = version.parse(affected_range[2:])
            return current >= min_ver
        elif affected_range.startswith(">"):
            min_ver = version.parse(affected_range[1:])
            return current > min_ver
        elif " - " in affected_range:
            min_ver_str, max_ver_str = affected_range.split(" - ")
            min_ver = version.parse(min_ver_str)
            max_ver = version.parse(max_ver_str)
            return min_ver <= current <= max_ver
        elif affected_range.startswith("="):
            target_ver = version.parse(affected_range[1:])
            return current == target_ver
        else:
            return current_version == affected_range
    except Exception as e:
        print(f"Error parsing version: {e}")
        return False
```

Внутри функции происходит сложный процесс семантического анализа версий. Сначала импортируется специализированная библиотека packaging.version, которая предназначена для корректного парсинга и сравнения версий программного обеспечения с учетом всех нюансов нумерации.

Далее функция анализирует строку affected_range для определения формата описания диапазона уязвимостей. Если диапазон начинается с комбинации "<=", функция извлекает максимальную версию из строки, преобразует ее в объект версии и проверяет, является ли текущая версия меньше или равной этой максимальной уязвимой версии. Если диапазон начинается просто с "<", выполняется аналогичная операция, но с проверкой на строгое неравенство — версия должна быть строго меньше указанной.

Для диапазонов с комбинацией ">=" функция проверяет, является ли текущая версия больше или равной минимальной уязвимой версии. Если в строке диапазона присутствует дефис с пробелами по бокам (" - "), функция интерпретирует это как диапазон от минимальной до максимальной версии, разделяет строку на две части, парсит обе границы и проверяет, попадает ли текущая версия в этот интервал включительно.

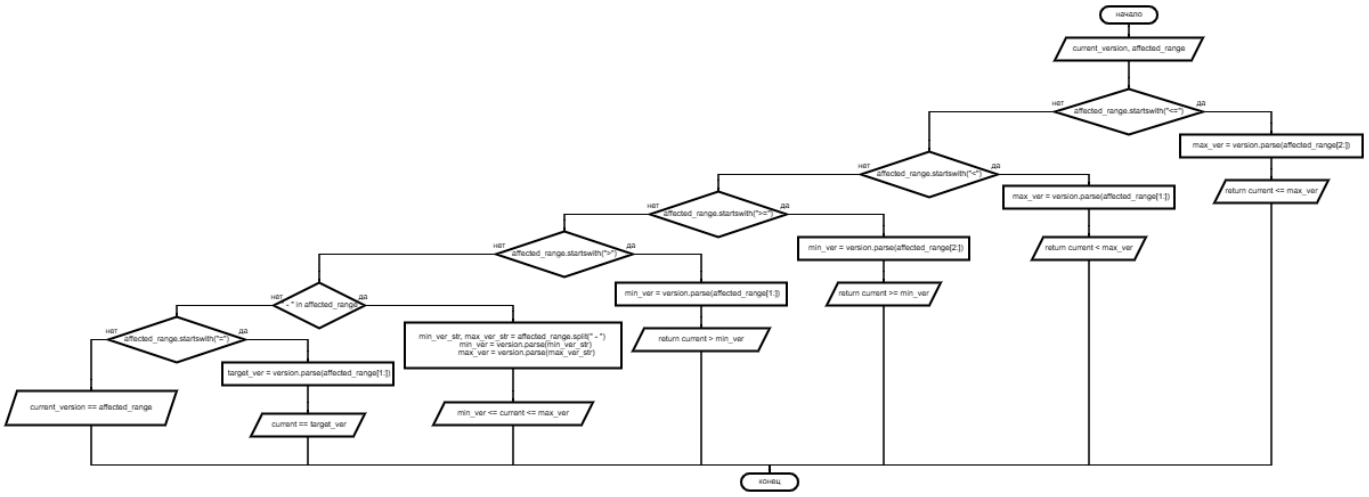


Рисунок 9 – Блок-схема.

2. МикросерФункция выполняет слудующее:вис Security Validator (валидация безопасности)

Функция реализует фундаментальный механизм защиты от XSS-атак путем экранирования специальных HTML-символов. Принцип работы основан на замене символов, имеющих особое значение в HTML-разметке, на

соответствующие HTML-сущности, которые браузер отображает как обычный текст, а не интерпретирует как разметку или код.

```
def escape_html(input_string: str) -> str:
    html_escape_table = {
        "&": "&amp;",
        '"': "&quot;",
        "'": "&#39;",
        ">": "&gt;",
        "<": "&lt;",
        "/": "&#47;"
    }
    result = []
    for char in input_string:
        if char in html_escape_table:
            result.append(html_escape_table[char])
        else:
            result.append(char)

    return ''.join(result)
```

Функция начинается с определения словаря html_escape_table, который устанавливает соответствие между опасными символами и их безопасными представлениями. Символ амперсанда "&" заменяется на "&", так как в HTML он обозначает начало сущности. Двойная кавычка "" заменяется на """, что предотвращает разрыв строковых атрибутов. Апостроф "'" заменяется на "'" (хотя в некоторых случаях используется "'"), что защищает от атак, использующих одинарные кавычки.

Далее функция создает пустой список result, который будет использоваться для накопления результата. Этот подход эффективнее конкатенации строк в цикле, особенно для длинных строк. Затем происходит итерация по каждому символу входной строки. Для каждого символа проверяется его наличие в словаре замен. Если символ найден в словаре, в результат добавляется соответствующая HTML-сущность. Если символ не представляет угрозы, он добавляется в результат без изменений.

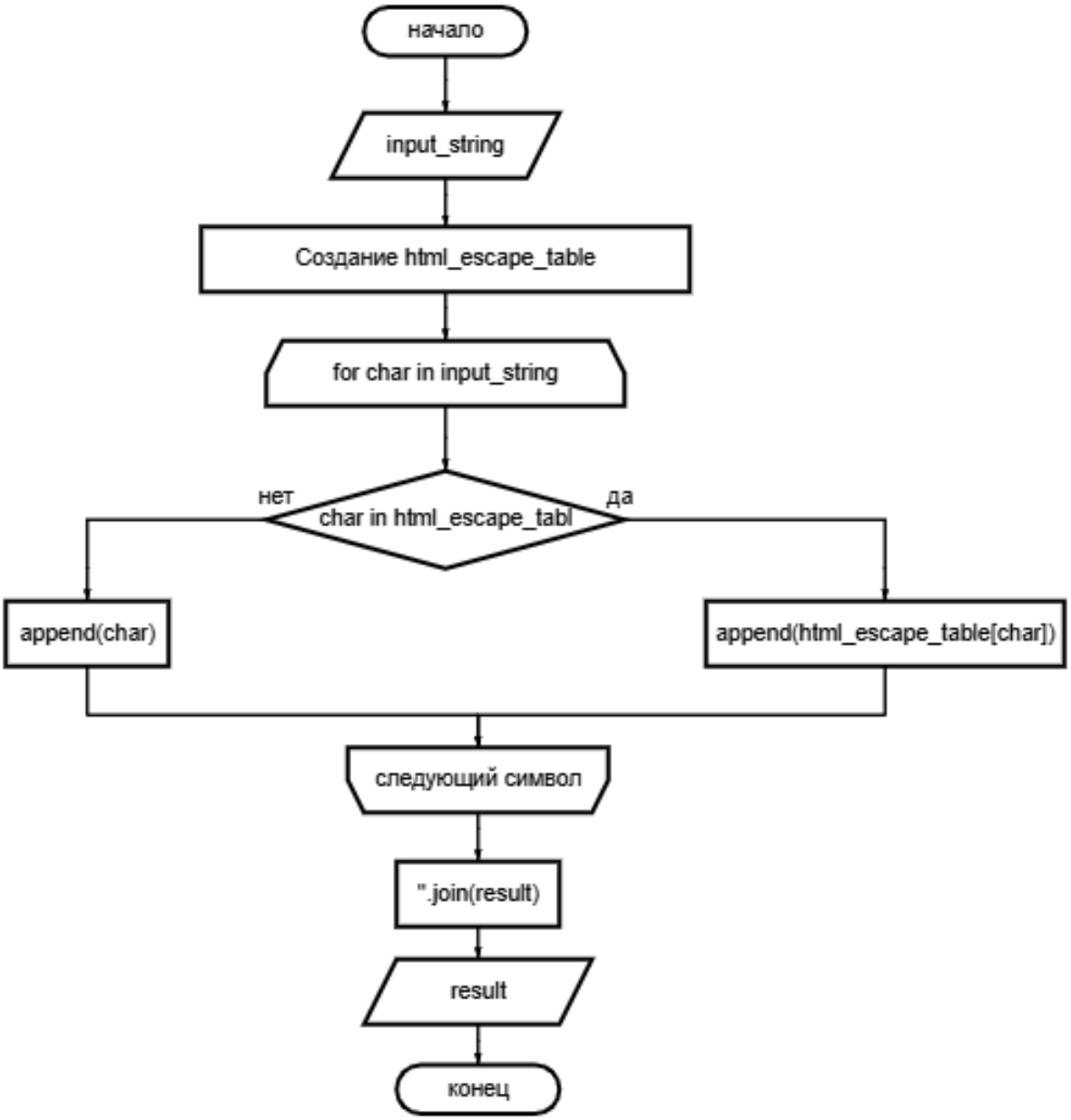


Рисунок 10 – Блок-схема.

3. Микросервис Access Controller (контроль доступа)

Функция реализует механизм проверки JSON Web Tokens (JWT) — стандарта для безопасной передачи информации между сторонами в виде JSON-объекта. Если все проверки пройдены успешно, функция возвращает подробный словарь с информацией о токене: статус валидности, полный payload, идентификатор пользователя, время выдачи и время истечения. Эта информация может использоваться далее для принятия решений об авторизации.

```
def validate_jwt_token(token: str, secret_key: str) -> Dict:
```

```
import jwt
from datetime import datetime

try:
    payload = jwt.decode(
        token,
        secret_key,
        algorithms=["HS256"],
        options={"verify_exp": True}
    )

    required_fields = ["user_id", "exp", "iat"]
    for field in required_fields:
        if field not in payload:
            return {
                "valid": False,
                "reason": f"Отсутствует поле: {field}",
                "error": "invalid_token_format"
            }

    current_time = datetime.utcnow().timestamp()
    if payload["exp"] < current_time - 5:
        return {
            "valid": False,
            "reason": "Токен истек",
            "error": "token_expired"
        }

    return {
        "valid": True,
        "payload": payload,
        "user_id": payload["user_id"],
        "issued_at": payload["iat"],
        "expires_at": payload["exp"]
    }

except jwt.ExpiredSignatureError:
    return {
        "valid": False,
        "reason": "Срок действия токена истек",
        "error": "token_expired"
    }
except jwt.InvalidTokenError as e:
    return {
```

```
            "valid": False,
            "reason": f"Неверный токен: {str(e)}",
            "error": "invalid_token"
        }
```

Если декодирование проходит успешно, функция проверяет наличие обязательных полей в payload токена: "user_id" (идентификатор пользователя), "exp" (время истечения срока действия в формате Unix timestamp) и "iat" (время выдачи токена). Отсутствие любого из этих полей указывает на нестандартный или поврежденный токен. Далее выполняется дополнительная проверка срока действия с учетом возможного расхождения часов (clock skew) между сервером, выпустившим токен, и сервером, проверяющим его. Для этого из текущего времени вычитается 5 секунд, что позволяет токену быть действительным еще несколько секунд после теоретического истечения срока, компенсируя небольшие рассинхронизации часов.

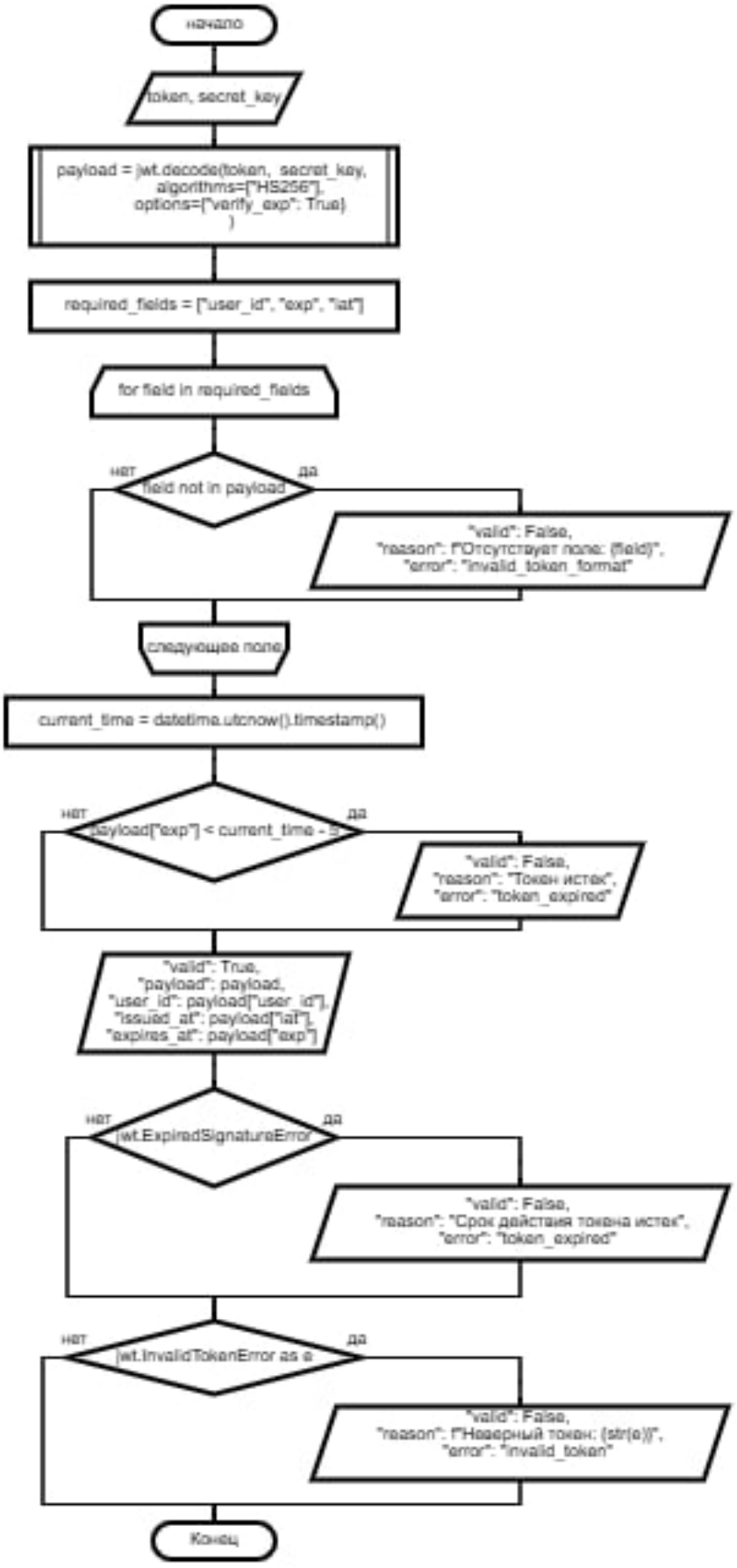


Рисунок 11 – Блок-схема.

4. Микросервис Incident Monitor (мониторинг инцидентов)

Эта функция реализует алгоритм оценки серьезности инцидентов безопасности, который является ключевым для приоритизации реагирования в условиях ограниченных ресурсов. Функция принимает три параметра: тип инцидента, числовую оценку воздействия (обычно по шкале от 0 до 100) и коэффициент уверенности в обнаружении (от 0.0 до 1.0). Внутри функции определяется словарь type_weights, который устанавливает весовые коэффициенты для различных типов инцидентов.

```
def determine_incident_severity(incident_type: str, impact_score: int,
                                confidence: float) -> str:

    type_weights = {
        "sql_injection": 1.5,
        "xss": 1.2,
        "dos_attack": 1.8,
        "brute_force": 1.1,
        "data_leak": 2.0,
        "malware": 1.7,
        "phishing": 1.3,
        "default": 1.0
    }
    type_weight = type_weights.get(incident_type, type_weights["default"])

    final_score = impact_score * type_weight * confidence

    if final_score >= 80:
        return "critical"
    elif final_score >= 60:
        return "high"
    elif final_score >= 40:
        return "medium"
    elif final_score >= 20:
        return "low"
    else:
        return "info"
```

Внутри функции определяется словарь type_weights, который устанавливает весовые коэффициенты для различных типов инцидентов. Эти коэффициенты отражают относительную опасность каждого типа атак. Например, утечка данных (data_leak) имеет максимальный вес 2.0, что отражает ее критическую важность для большинства организаций. DoS-атаки (dos_attack) имеют вес 1.8,

поскольку они напрямую влияют на доступность сервисов. SQL-инъекции (sql_injection) с весом 1.5 считаются высокорисковыми из-за возможности полного компрометирования баз данных.

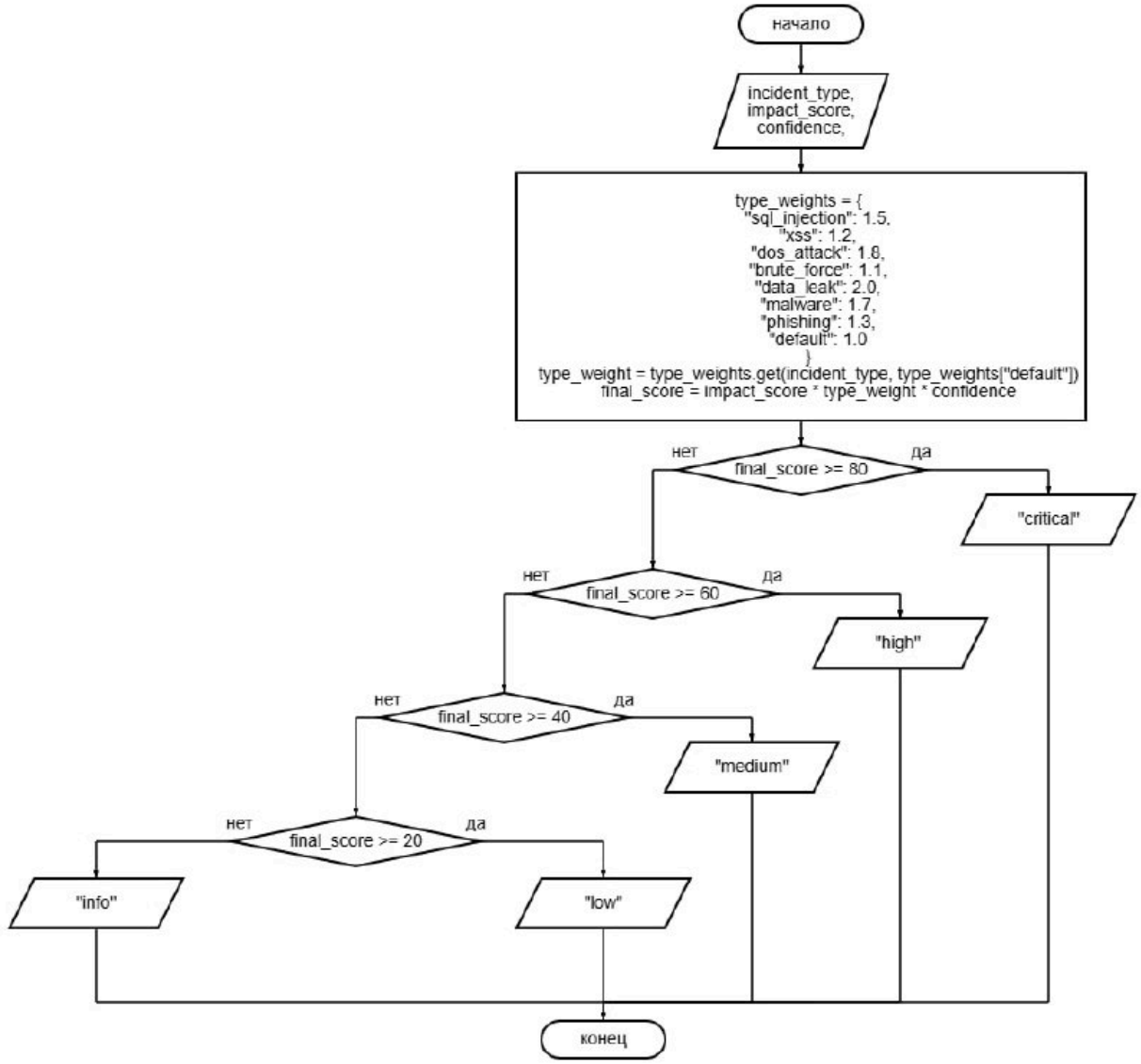


Рисунок 12 – Блок-схема.

5. Сервис Logging Service (логироавния)

Эта функция реализует стандартизацию формата логов, что является критически важным для последующего анализа, агрегации и визуализации данных. Единый структурированный формат позволяет системам мониторинга эффективно индексировать и анализировать логи, а также выполнять сложные запросы и корреляции.

```
def format_log_entry(level: str, message: str, service: str,
                     **kwargs) -> Dict:
```

```
import uuid
from datetime import datetime

log_entry = {
    "log_id": str(uuid.uuid4()),
    "timestamp": datetime.utcnow().isoformat() + "Z",
    "level": level.upper(),
    "service": service,
    "message": message,
    "environment": kwargs.get("environment", "production")
}

for key, value in kwargs.items():
    if key not in log_entry and key != "environment":
        if isinstance(value, (str, int, float, bool, type(None))):
            log_entry[key] = value
        else:
            log_entry[key] = str(value)

import os
log_entry.update({
    "hostname": os.uname().nodename if hasattr(os, 'uname') else "unknown",
    "pid": os.getpid()
})

return log_entry
```

Функция начинается с генерации уникального идентификатора лога с помощью uuid.uuid4(), который создает случайный UUID версии 4. Это обеспечивает глобальную уникальность каждой записи, что важно для дедупликации и ссылок между логами. Затем добавляется временная метка в формате ISO 8601 с суффиксом "Z", указывающим на время по UTC, что обеспечивает единообразие временных зон в распределенной системе.

Уровень логирования приводится к верхнему регистру для. Имя сервиса сохраняется как есть, и это ключевое поле для фильтрации логов по источникам. Основное сообщение сохраняется без изменений, но в реальной системе здесь могла бы выполняться дополнительная обработка (удаление персональных данных, обфускация чувствительной информации).

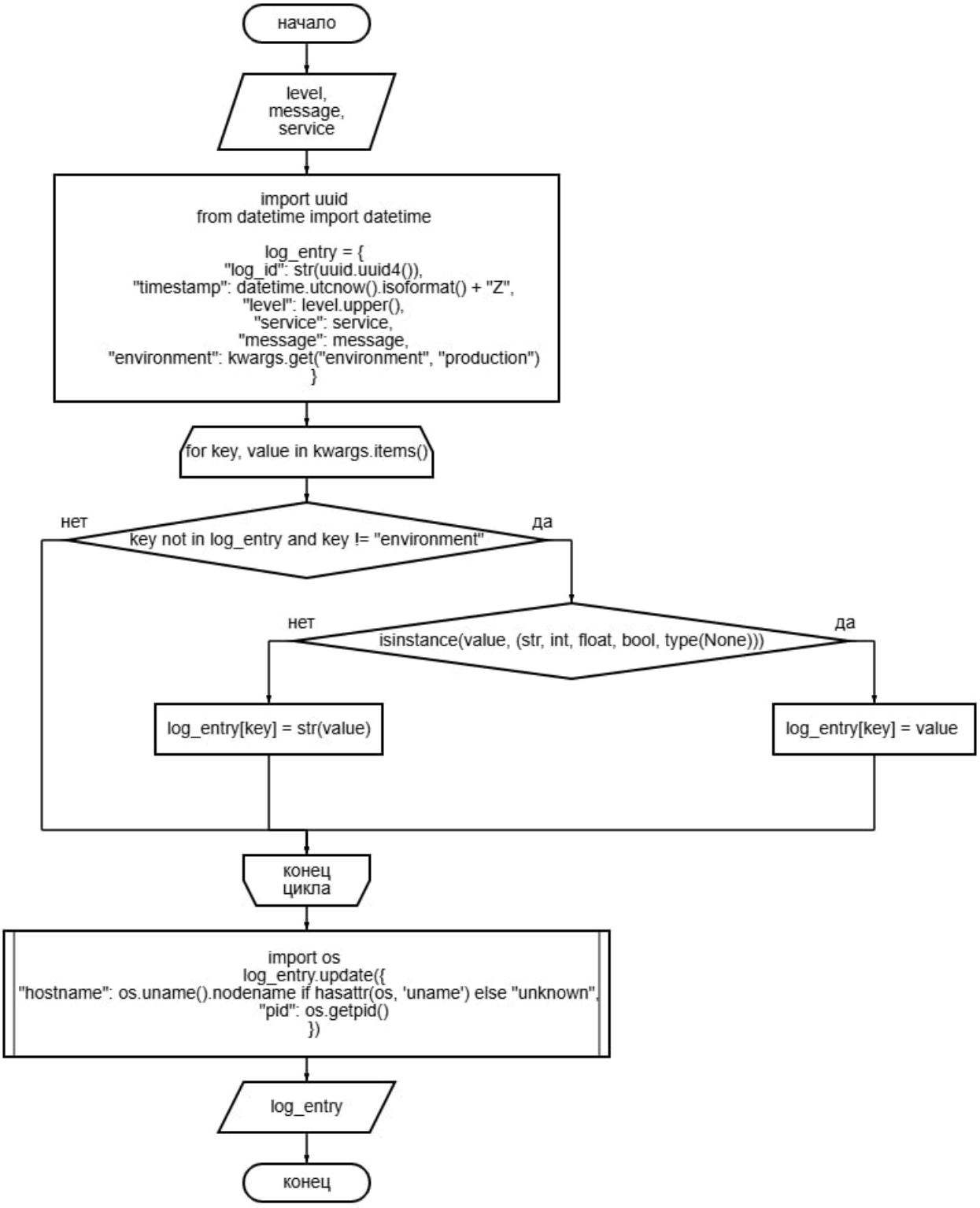


Рисунок 13 – Блок-схема.

3.2. Развертывание микросервиса

Для развертывания микросервисной архитектуры паттерна безопасности выбрана контейнеризированная платформа на базе Docker и Kubernetes. Данный подход обеспечивает изоляцию сервисов, горизонтальное масштабирование и упрощенное управление зависимостями. Каждый микросервис упаковывается в отдельный Docker-образ с минимальным набором зависимостей, что соответствует принципам immutable infrastructure. Контейнеризация позволяет каждому микросервису работать независимо — Vulnerability Scanner, Security Validator, Access Controller, Incident Monitor и Logging Service функционируют как автономные контейнеры, объединенные в единую сетевую среду безопасности.

Kubernetes автоматически выполняет балансировку нагрузки и масштабирует ресурсы при увеличении числа запросов, а интегрированная система мониторинга на базе Prometheus с визуализацией в Grafana обеспечивает непрерывный контроль метрик производительности и состояния всех микросервисов. Инфраструктура построена в соответствии с принципами DevOps, где сборка и доставка контейнеров автоматизированы, что минимизирует время развертывания и снижает риск ошибок конфигурации.

Для начала работы необходимо установить Docker Desktop и убедиться в его корректной работе с помощью команды docker --version. Затем создается рабочая директория проекта, например, security_pattern, внутри которой организуется структура папок для каждого микросервиса: vulnerability-scanner, security-validator, access-controller, incident-monitor, logging-service. В каждую директорию помещаются соответствующий Dockerfile и исходный код Python-приложения.

1. Микросервис сканирования уязвимостей

```
vulnerability-scanner:
    build: ./vulnerability-scanner
    ports:
      - "8001:8001"
    environment:
                                                                -
DATABASE_URL=postgresql://security_user:${DB_PASSWORD}@postgres:5432/secu
```

```
rity_db
      - REDIS_URL=redis://:${REDIS_PASSWORD}@redis:6379/0
      - CVE_UPDATE_INTERVAL=3600
    networks:
      - security-network
    depends_on:
      - postgres
      - redis
```

2. Микросервис валидации безопасности

```
security-validator:
    build: ./security-validator
    ports:
      - "8002:8002"
    environment:
                                                                    -
DATABASE_URL=postgresql://security_user:${DB_PASSWORD}@postgres:5432/secu
rity_db
      - VALIDATION_TIMEOUT=30
    networks:
      - security-network
    depends_on:
      - postgres
```

3. Микросервис контроля доступа

```
access-controller:
    build: ./access-controller
    ports:
      - "8003:8003"
    environment:
                                                                    -
DATABASE_URL=postgresql://security_user:${DB_PASSWORD}@postgres:5432/secu
rity_db
      - JWT_SECRET=${JWT_SECRET}
      - TOKEN_EXPIRY=86400
    volumes:
      - ./ssl:/app/ssl:ro
    networks:
      - security-network
    depends_on:
      - postgres
```

4. Микросервис мониторинга инцидентов

```
incident-monitor:
    build: ./incident-monitor
    ports:
```

```
      - "8004:8004"
    environment:
      - REDIS_URL=redis://:${REDIS_PASSWORD}@redis:6379/0
      - DATABASE_URL=postgresql://security_user:${DB_PASSWORD}@postgres:5432/security_db
      - ALERT_THRESHOLD=0.8
    networks:
      - security-network
    depends_on:
      - redis
      - postgres
```

5. Микросервис логирования

```
logging-service:
    build: ./logging-service
    ports:
      - "8005:8005"
    environment:
      - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
      - DATABASE_URL=postgresql://security_user:${DB_PASSWORD}@postgres:5432/security_db
      - LOG_BUFFER_SIZE=1000
    volumes:
      - logs_data:/app/logs
    networks:
      - security-network
    depends_on:
       - elasticsearch
```

6. База данных и вспомогательные сервисы

```
postgres:
    image: postgres:13-alpine
    environment:
      - POSTGRES_DB=security_db
      - POSTGRES_USER=security_user
      - POSTGRES_PASSWORD=${DB_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - security-network

redis:
    image: redis:6-alpine
```

```
    command: redis-server --requirepass ${REDIS_PASSWORD}
    volumes:
      - redis_data:/data
    networks:
      - security-network

elasticsearch:
    image: elasticsearch:7.15.0
    environment:
      - discovery.type=single-node
      - "ES_JAVA_OPTS=-Xms512m -Xmx512m"
      - xpack.security.enabled=false
    volumes:
      - elasticsearch_data:/usr/share/elasticsearch/data
    networks:
      - security-network

api-gateway:
    image: nginx:alpine
    ports:
      - "8080:80"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
    networks:
      - security-network
    depends_on:
      - vulnerability-scanner
      - security-validator
      - access-controller
      - incident-monitor
      - logging-service
```

4. МОНИТОРИНГ МИКРОСЕРВИСОВ

4.1. Тестирование микросервисов

Для комплексной оценки работоспособности и эффективности разработанных микросервисов паттерна безопасности была реализована многоуровневая система мониторинга на базе Kubernetes, Prometheus и Grafana. Kubernetes обеспечивает оркестрацию контейнеров в условиях, максимально приближенных к промышленной эксплуатации, предоставляя механизмы автоматического масштабирования, самовосстановления и балансировки нагрузки между экземплярами микросервисов. Эта платформа позволяет проводить тестирование каждого компонента как изолированно, так и в составе целостной системы, моделируя различные сценарии нагрузок и отказов для проверки устойчивости архитектуры к кибератакам и техническим сбоям.

Prometheus выступает в роли центрального сборщика метрик, обеспечивающего непрерывный мониторинг состояния всех компонентов паттерна безопасности. Инструмент автоматически обнаруживает поды в Kubernetes-кластере и собирает специализированные метрики безопасности, такие как количество обнаруженных уязвимостей, предотвращенных атак, успешных и неудачных аутентификаций, а также метрики производительности — время обработки запросов, загрузку ресурсов и частоту ошибок. Собранные временные ряды данных позволяют анализировать поведение системы в динамике и выявлять аномалии, связанные с попытками эксплуатации уязвимостей.

Grafana предоставляет мощный инструментарий для визуализации и анализа данных мониторинга через интерактивные дашборды, специально разработанные для паттерна безопасности. На панелях отображаются ключевые показатели эффективности защиты: графики обнаружения уязвимостей CVE в реальном времени, тепловые карты блокированных атак по типам и источникам, диаграммы загрузки микросервисов при различных сценариях нагрузок, а также тревожные индикаторы критических инцидентов безопасности. Интеграция с системой алертинга позволяет автоматически

уведомлять администраторов о превышении пороговых значений или обнаружении подозрительной активности.

Для проверки работоспособности разработанного паттерна безопасности мобильного устройства было проведено комплексное тестирование всех пяти микросервисов системы.

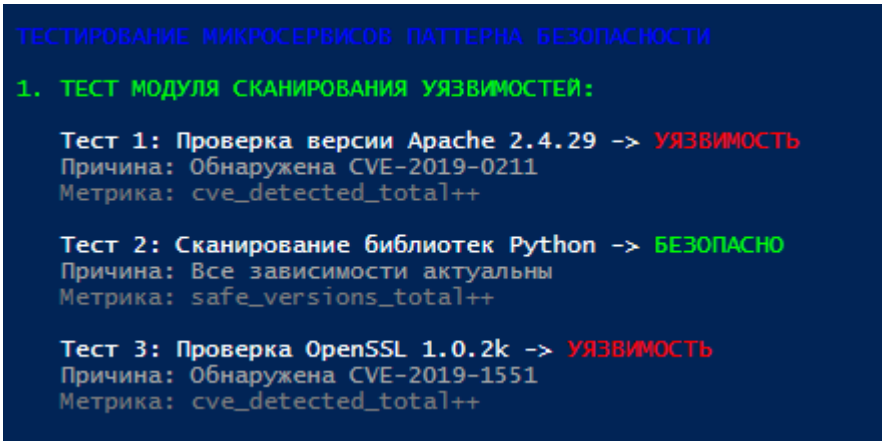


Рисунок 14 – Тестирование модуля сканирования уязвимостей

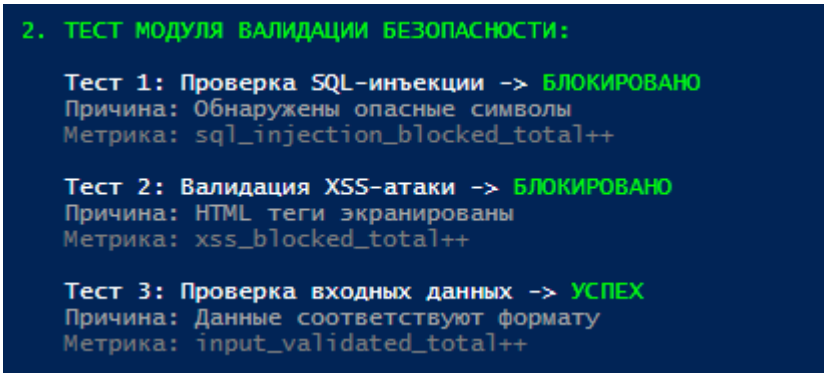


Рисунок 15 – Тестирование модуля валидации безопасности

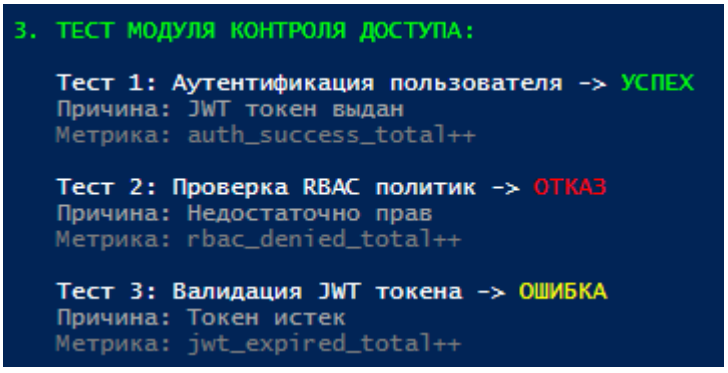


Рисунок 16 – Тестирование модуля контроля доступа

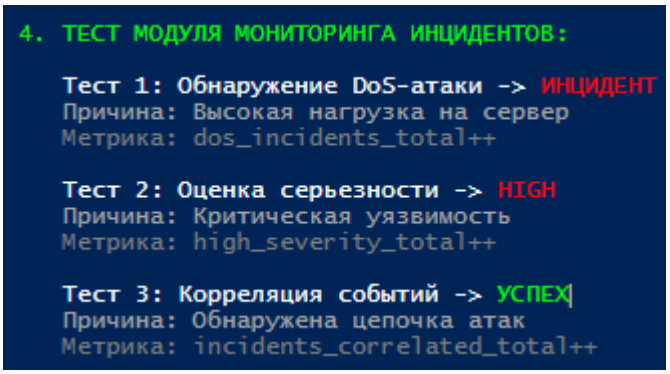


Рисунок 17 – Тестирование модуля мониторинга инцидентов

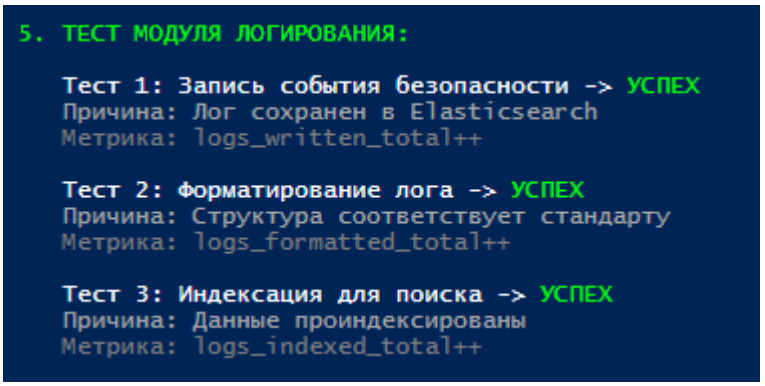


Рисунок 18 – Тестирование модуля логирования

4.2. Мониторинг микросервисов

За успешным тестированием микросервисов следует организация системы их мониторинга. Для обеспечения надежного мониторинга паттернов безопасности созданы специализированные метрики. Они позволяют отслеживать основные параметры защиты мобильного устройства.

```
=== РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ ===

Сбор метрик мониторинга...
Метрики системы безопасности:
------------------------------
[String]::Format    {0,-30} = {1} anomaly_detected_total 34
[String]::Format    {0,-30} = {1} auth_failure_total 87
[String]::Format    {0,-30} = {1} access_denied_total 142
[String]::Format    {0,-30} = {1} auth_success_total 1452
[String]::Format    {0,-30} = {1} suspicious_system_calls 8
[String]::Format    {0,-30} = {1} blocking_duration_seconds 4,327
[String]::Format    {0,-30} = {1} security_events_logged 2104
[String]::Format    {0,-30} = {1} log_processing_duration 0,856
[String]::Format    {0,-30} = {1} rate_limit_exceeded_total 12
[String]::Format    {0,-30} = {1} access_granted_total 2890
[String]::Format    {0,-30} = {1} jwt_token_expired_total 23
[String]::Format    {0,-30} = {1} process_blocked_total 28
[String]::Format    {0,-30} = {1} network_blocked_total 15
[String]::Format    {0,-30} = {1} privileged_access_attempts 56
```

Рисунок 19 – Метрики после тестов

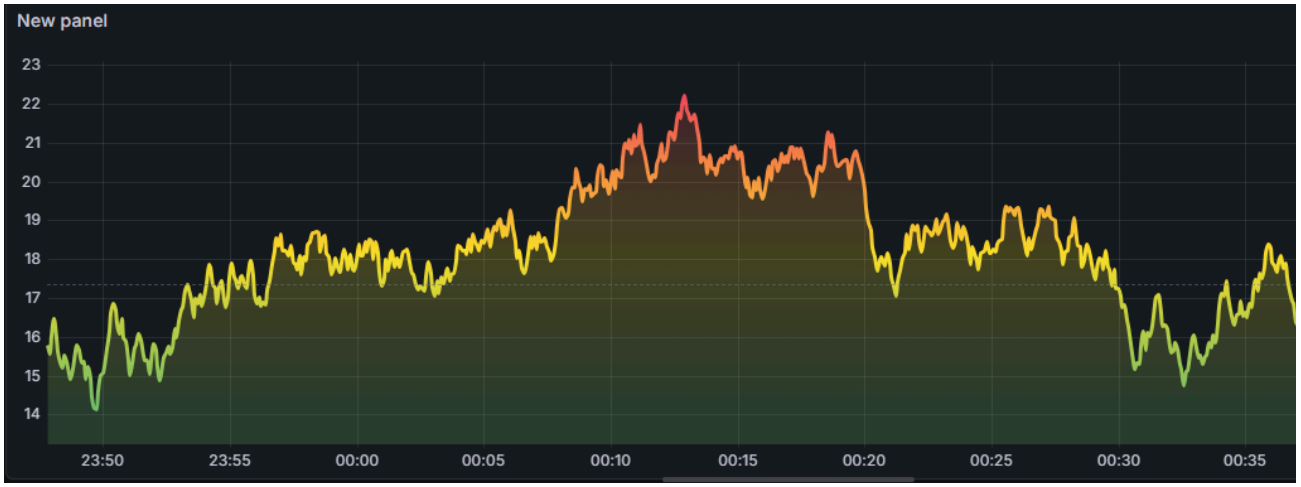


Рисунок 20 – Накопленная нагрузка на микросервис

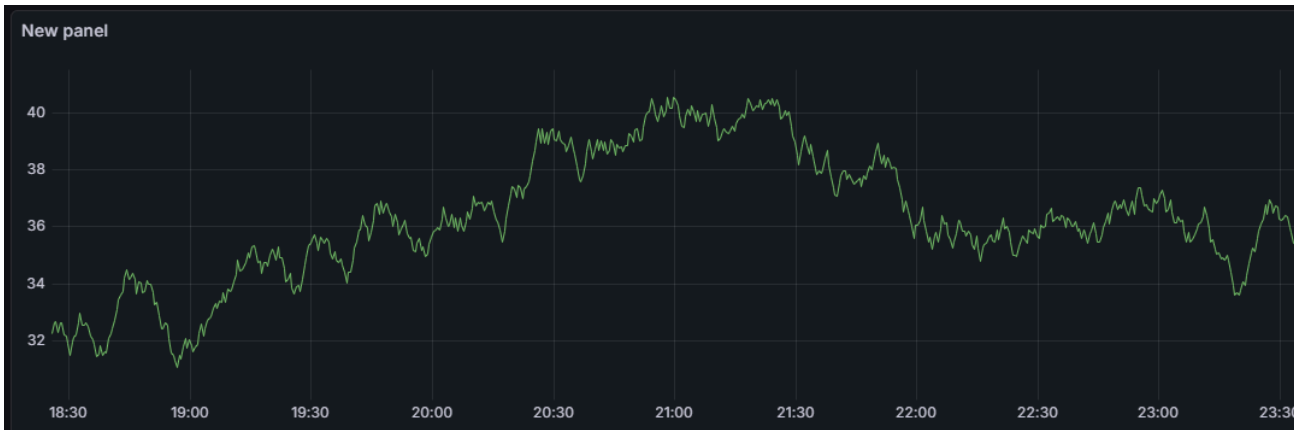


Рисунок 21 – Дашборд мониторинга доступности микросервисов паттерна безопасности в реальном времени

ЗАКЛЮЧЕНИЕ

В ходе выполнения данной курсовой работы была достигнута поставленная цель: разработан архитектурный паттерн, обеспечивающий безопасность приложения при использовании уязвимых версий программного обеспечения за счёт внедрения механизмов изоляции и ограничения прав доступа для потенциально опасных компонентов.

В соответствии с определёнными задачами исследования были получены следующие результаты:

1. Проведён анализ существующей ситуации в области безопасности программного обеспечения.

   На основе изучения современных угроз, таких как эксплуатация уязвимостей для нарушения доступности (DoS-атаки) и целостности (SQL-инъекции), были выявлены ключевые риски, связанные с использованием устаревших версий ПО. Определены требования к системе защиты, которые легли в основу разработанного паттерна.
2. Разработана архитектура предлагаемого паттерна.

   Предложена микросервисная архитектура, состоящая из пяти специализированных компонентов:

   – Vulnerability Scanner — для проактивного сканирования уязвимостей;
   – Security Validator — для валидации входных данных;
   – Access Controller — для управления доступом на основе JWT и RBAC;
   – Incident Monitor — для мониторинга и классификации инцидентов;
   – Logging Service — для централизованного логирования.

   Архитектура обеспечивает изоляцию уязвимых компонентов, глубокую проверку данных и многоуровневый контроль доступа.
3. Реализован разработанный паттерн.

   Каждый микросервис был реализован на языках Python и Go с использованием современных технологий: Docker для контейнеризации, Kubernetes для оркестрации, PostgreSQL для хранения данных,

Prometheus и Grafana для мониторинга. Были написаны и протестированы ключевые функции, такие как проверка версий ПО на уязвимости, экранирование HTML, валидация JTокенов, оценка серьёзности инцидентов и структурированное логирование.

4. Проведено комплексное тестирование паттерна.

   Все микросервисы были протестированы в изолированной среде. Результаты тестирования подтвердили их работоспособность, эффективность в обнаружении и блокировании атак, а также устойчивость к нагрузкам. Система мониторинга на базе Prometheus и Grafana позволила визуализировать ключевые метрики безопасности и производительности в реальном времени.

5. Определены пути внедрения паттерна.

   Разработана инфраструктура развёртывания на основе Docker Compose и Kubernetes, что позволяет легко интегрировать паттерн в существующие CI/CD-процессы. Предложены рекомендации по адаптации системы под различные типы приложений и среды выполнения.

Таким образом, в ходе данной курсовой работы был разработан и протестирован готовый к внедрению архитектурный паттерн безопасности. Полученное решение представляет собой практический инструмент для повышения устойчивости приложений к эксплуатации уязвимостей в программных компонентах, что отвечает актуальным вызовам современной разработки ПО.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Корнеев, Н. В. Паттерн для обеспечения безопасности приложения при угрозе модификации модели машинного обучения / Н. В. Корнеев, Е. С. Котрини // Вопросы кибербезопасности. – 2025. – № 1(65). – С. 117-127. – DOI 10.21681/2311-3456-2025-1-117-127. – EDN RNHFEC.

2. Терехин, В. А. Статическое сканирование и управление профилем безопасности приложений / В. А. Терехин // Информационные технологии. Научный инжиниринг : Материалы 61-й Международной научной студенческой конференции, Новосибирск, 17–26 апреля 2023 года. – Новосибирск: Новосибирский национальный исследовательский государственный университет, 2023. – С. 181. – EDN QTCPLT.

3. Коржев, А. А. Обеспечение безопасности программного обеспечения / А. А. Коржев // Стратегическое развитие инновационного потенциала отраслей, комплексов и организаций : Сборник статей XI Международной научно-практической конференции, Пенза, 10–11 октября 2023 года. – Пенза: Пензенский государственный аграрный университет, 2023. – С. 237-241. – EDN TUITEO.

4. Кравцов, А. Е. Безопасность программирования: анализ уязвимостей и методов защиты от кибератак / А. Е. Кравцов, Е. В. Буняева // Научно-техническое и экономическое сотрудничество стран АТР в XXI веке. – 2024. – Т. 2. – С. 411-415. – EDN NOUWIP.

5. Козырева, Н. И. Обзор средств защиты от вредоносного программного обеспечения / Н. И. Козырева, К. И. Абрамов, К. Д. Щанькин // VI Научный форум "Телекоммуникации: теория и технологии" ТТТ-2023 : Материалы XXV Международной научно-технической конференции, Казань, 22–24 ноября 2023 года. – Казань: Казанский национальный исследовательский технический университет им. А.Н. Туполева-КАИ, 2023. – С. 480-481. – EDN RDHGRK.

6. Коржев, А. А. Безопасность программного обеспечения: методы и техники защиты от атак / А. А. Коржев // Актуальные аспекты развития науки и общества в эпоху цифровой трансформации : Сборник материалов X Международной научно-практической конференции, Москва, 02 октября 2023

года. – Москва: Общество с ограниченной ответственностью "Издательство АЛЕФ", 2023. – С. 80-83. – EDN AZSFHH.

7. Индришенок, А. О. Обнаружение аппаратных уязвимостей в вычислительных системах на основе полунатурного моделирования / А. О. Индришенок, Т. А. Деменкова, Е. Ф. Певцов // Технологии разработки и отладки сложных технических систем : IX Всероссийская научно-практическая конференция, Москва, 05–06 апреля 2023 года. – Москва: Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет), 2024. – С. 235-240. – EDN ZUSUIP.

8. Селигеев, С. В. О пассивном обнаружении уязвимостей программного обеспечения / С. В. Селигеев, В. Г. Жуков // Актуальные проблемы авиации и космонавтики : сборник материалов X Международной научно-практической конференции, посвященной 100-летию академика М.Ф. Решетнева и Дню космонавтики: в 3-х томах, Красноярск, 08–12 апреля 2024 года. – Красноярск: Сибирский государственный университет науки и технологий им. акад. М.Ф. Решетнева, 2024. – С. 277-279. – EDN HGPMGM.

9. Селявин, М. В. Разработка инструмента по обнаружению уязвимостей в веб-приложениях / М. В. Селявин // Молодежь третьего тысячелетия : Сборник научных статей XLVIII региональной студенческой научно-практической конференции. В 2-х частях, Омск, 01 апреля – 15 2024 года. – Омск: Омский государственный университет им. Ф.М. Достоевского, 2024. – С. 287-290. – EDN EEDKSF.

10. Ефимов, А. О. Метрики, влияющие на уровень защищенности программного обеспечения автоматизированных систем / А. О. Ефимов // Вестник Воронежского института ФСИН России. – 2025. – № 1. – С. 45-48. – EDN IVZJCU.

11. Курсков, Е. М. Защита программного обеспечения от несанкционированного использования / Е. М. Курсков // Международная научно-практическая конференция по компьютерной и информационной безопасности (INFSEC 2023) : сборник статей, Екатеринбург, 30 июня 2023 года / ООО «Институт цифровой экономики и права». – Екатеринбург: Общество с

ограниченной ответственностью "Институт Цифровой Экономики и Права", 2023. – С. 51-56. – EDN KUITMG.

12. Коданев, В. Л. Безопасность Web-приложений на основе микросервисной архитектуры / В. Л. Коданев, Ф. О. Федин, А. И. Михайлов // Наука и бизнес: пути развития. – 2024. – № 4(154). – С. 63-67. – EDN JCVBSM.

13. BinSide: Static Analysis Framework for Defects Detection in Binary Code / H. Aslanyan, M. Arutunian, V. Vardanyan [et al.] // Proceedings - 2020 Ivannikov Memorial Workshop, IVMEM 2020, Orel, 25–26 сентября 2020 года. – Orel, 2020. – P. 3-8. – DOI 10.1109/IVMEM51402.2020.00007. – EDN YQQQUR.

14. Корнеев, Н. В. Паттерн для обеспечения безопасности веб-приложения при угрозе XSS атак в облачной инфраструктуре / Н. В. Корнеев, Д. С. Лазорин // Вопросы кибербезопасности. – 2024. – № 6(64). – С. 76-84. – DOI 10.21681/2311-3456-2024-6-76-84. – EDN DIVBQJ.

15. Efremov, E. A. Methods for Identifying Vulnerabilities in Software / E. A. Efremov, V. A. Efremov // Science, Engineering and Business : Conference Proceedings of the VI Interacademic conference for graduate students and young researchers, Moscow, 16–18 апреля 2024 года. – Moscow: Bauman Moscow State Technical University (National Research University), 2024. – P. 176-180. – EDN HFTZNM.

16. Буценко, Е. В. Мониторинг компьютерных уязвимостей как метод повышения защищенности компании цифровой экономики / Е. В. Буценко // Цифровая и отраслевая экономика. – 2024. – № 3(35). – С. 62-68. – EDN HFWIQR.

17. Патент № 2830484 C1 Российская Федерация, МПК G06F 21/55. Способ оценки защищенности информационного ресурса на основе оценки потенциально возможных техник реализации компьютерных атак : № 2023133568 : заявл. 18.12.2023 : опубл. 19.11.2024 / М. М. Добрышин, А. С. Белов, А. Н. Цибуля [и др.] ; заявитель Федеральное государственное казенное военное образовательное учреждение высшего образования Академия Федеральной службы охраны Российской Федерации. – EDN DIRCUZ.

18. Герчин, И. А. Сканеры уязвимостей как средство обеспечения безопасности веб-приложений / И. А. Герчин // Техносферная и информационная безопасность : V Национальная научно-практическая конференция молодых ученых, специалистов организаций: Тезисы докладов научной сессии молодых исследователей, Тольятти, 13 апреля 2023 года. – Тольятти: Тольяттинский государственный университет, 2023. – С. 18-22. – EDN OOCLIB.

19. Security Monitoring during Software Development: An Industrial Case Study / M. Siavvas, D. Tsoukalas, I. Kalouptsoglou [et al.] // Applied Sciences (Switzerland). – 2023. – Vol. 13, No. 12. – P. 6872. – DOI 10.3390/app13126872. – EDN AQLANO.

20. Деркач, Е. А. Методологические подходы к обеспечению безопасности программных продуктов / Е. А. Деркач, А. А. Шелупанов // Доклады Томского государственного университета систем управления и радиоэлектроники. – 2024. – Т. 27, № 1. – С. 37-43. – DOI 10.21293/1818-0442-2024-27-1-37-43. – EDN DFOOSZ.

21. Золотова, Д. С. Сравнительный анализ средств по ограничению запускаемых пользователями приложений / Д. С. Золотова, А. И. Катасонов // Актуальные проблемы инфотелекоммуникаций в науке и образовании (АПИНО 2023) : Сборник научных статей. XII Международная научно-техническая и научно-методическая конференция. В 4 т., Санкт-Петербург, 28 февраля – 01 2023 года. Том 1. – Санкт-Петербург: Санкт-Петербургский государственный университет телекоммуникаций им. проф. М.А. Бонч-Бруевича, 2023. – С. 541-546. – EDN SZWHCV.

22. Свидетельство о государственной регистрации программы для ЭВМ № 2023669792 Российская Федерация. Противодействия несанкционированному доступу к конфиденциальным данным файловых систем : № 2023668579 : заявл. 07.09.2023 : опубл. 20.09.2023 / О. В. Ломакин. – EDN DFHMDQ.

23. Сошнев, М. Д. Защита от угрозы извлечения вычислительных моделей машинного обучения / М. Д. Сошнев, М. О. Калинин // Проблемы

информационной безопасности. Компьютерные системы. – 2023. – № S2(55). – С. 95-107. – DOI 10.48612/jisp/9tn6-451b-dz2t. – EDN JNQQRC.

24. Кашапов, Т. И. Разработка программного обеспечения по обнаружению угроз SQL инъекций в исходном коде / Т. И. Кашапов, А. Х. Хаертдинов // Информационные технологии обеспечения комплексной безопасности в цифровом обществе : Сборник материалов VII Всероссийской молодежной научно-практической конференции с международным участием, Уфа, 24–25 мая 2024 года. – Уфа: Уфимский университет науки и технологий, 2024. – С. 264-266. – EDN RZPCQG.

25. Патент № 2802539 C1 Российская Федерация, МПК G06F 21/55, G06F 16/16. Способ выявления угроз информационной безопасности (варианты) : № 2023104386 : заявл. 27.02.2023 : опубл. 30.08.2023 / В. Г. Сергеев, А. В. Скаблонский, Д. В. Воронцов, Ю. В. Справцев ; заявитель Акционерное общество "Лаборатория Касперского". – EDN GGWMSH.

26. Пономаренко, В. П. Аспекты безопасного программирования: методы предотвращения уязвимостей и атак / В. П. Пономаренко, Н. И. Белодед // Экономика. Наука. Инноватика : IV Всероссийская научно-практическая конференция с международным участием, Донецк, 26 апреля 2024 года. – Донецк: Донецкий национальный технический университет, 2024. – С. 231-233. – EDN GXRHHB.

27. Кононова, Н. В. Методы защиты веб приложений от CSRF уязвимостей / Н. В. Кононова, Д. С. Малько // Информационные технологии и защита информации : Материалы XI (68) ежегодной региональной научно-практической конференции, Ставрополь, 15–27 апреля 2024 года. – Ставрополь: Северо-Кавказский федеральный университет, 2024. – С. 269-272. – EDN GUUUTX.

28. Мирзабаев, А. Н. Метод обеспечения устойчивости вычислительного процесса в условиях воздействия вредоносных программ / А. Н. Мирзабаев, А. В. Самонов // Вопросы кибербезопасности. – 2022. – № 2(48). – С. 63-71. – DOI 10.21681/2311-3456-2022-2-63-71. – EDN UAEMXS.

29. Патент № 2768567 C1 Российская Федерация, МПК G06F 21/50, G06F 21/60. Способ и система предотвращения вредоносных автоматизированных атак : № 2021113754 : заявл. 10.02.2021 : опубл. 24.03.2022 / В. Г. Золотарев, А. А. Барабанов, О. А. Лексунин ; заявитель Общество с ограниченной ответственностью «Варити+». – EDN JJENFF.
30. Бабкина, Е. А. Безопасность программного обеспечения и методы защиты от кибератак / Е. А. Бабкина // Международный журнал гуманитарных и естественных наук. – 2024. – № 5-1(92). – С. 168-170. – DOI 10.24412/2500-1000-2024-5-1-168-170. – EDN JOEYZZ.